

Database Indexing

RDBMS — Complete Study Notes

Data Pages · B+ Tree · Clustered Index · Non-Clustered Index
Page Splits · Block Mapping · NoSQL Indexing · MongoDB · Cassandra · DynamoDB

TOPICS COVERED

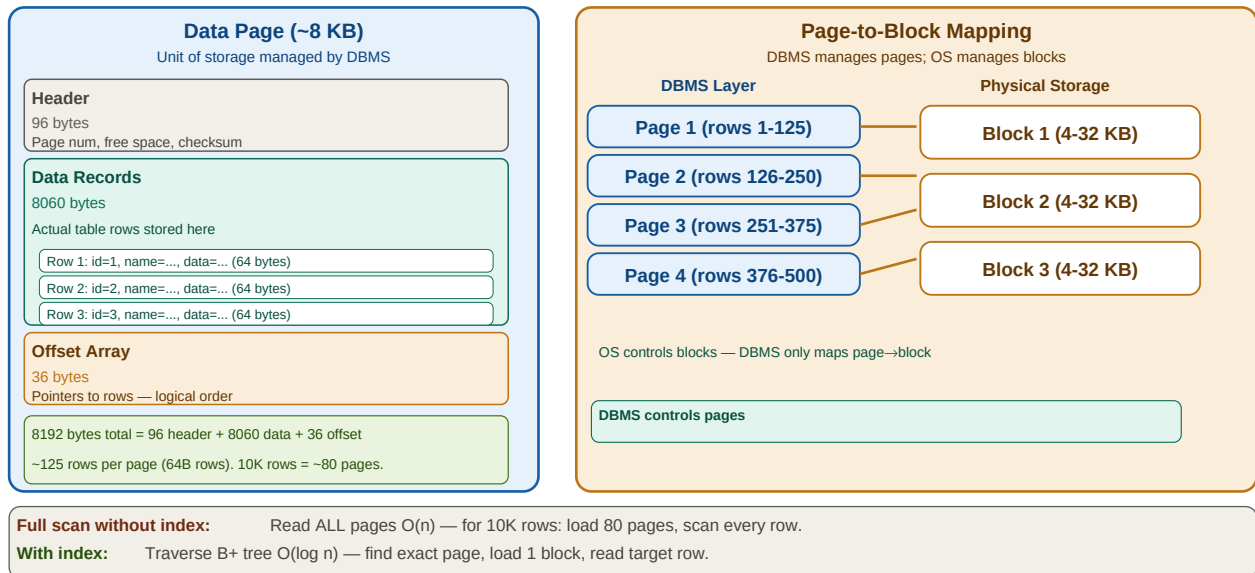
1. How Table Data is Physically Stored
2. Data Pages — Header, Records, Offset Array
3. Page-to-Block Mapping
4. B+ Tree — The Index Data Structure
5. Clustered Index — Deep Dive
6. Non-Clustered Index — Deep Dive
7. Data Insertion Flow & Page Splits
8. Index Overhead & Design Rules

1. How Table Data is Physically Stored	3
2. Data Page Structure — Header, Records, Offset Array	3
3. Page-to-Block Mapping	3
4. Why Indexing?	4
5. B+ Tree — The Index Data Structure	4
6. Clustered Index	5
7. Non-Clustered Index	5
8. Query Execution Steps with Indexing	6
9. Data Insertion Flow & Page Splits	6
10. NoSQL Indexing — MongoDB, Cassandra, DynamoDB, Redis	7
11. Index Overhead, Design Rules & Bonus Concepts	8
12. Quick Revision & Interview Questions	8

1, 2 & 3. Physical Storage — Data Pages & Block Mapping

Every table you create in an RDBMS is stored physically as **Data Pages** — not as the rows-and-columns view you see in SQL. A data page is typically **8 KB (8192 bytes)** and is the fundamental unit the DBMS reads and writes. Understanding this is essential to understanding why indexing works.

Physical Storage — Data Page Structure Page-to-Block Mapping



Data page: 96B header + 8060B data records + 36B offset array = 8192B · ~125 rows at 64 bytes each · Page-to-block mapping on right

Section	Size	Contents	Purpose
Header	96 bytes	Page number, free space, checksum	DBMS metadata for page management
Data Records	8060 bytes	Actual table rows stored here	The real data — rows packed in sequence
Offset Array	36 bytes	Pointers (offsets) to each row in the page	Enables $O(1)$ access to any row by position
Data Page total	~8 KB	All three sections combined	Unit of I/O — DBMS reads/writes one page at a time
Data Block	4-32 KB	Physical disk unit holding one or more pages	OS/storage manages this — DBMS only maps page→block

A table with 10,000 rows at 64 bytes each needs: $10000 * 64 = 640,000$ bytes / 8060 = ~80 data pages.

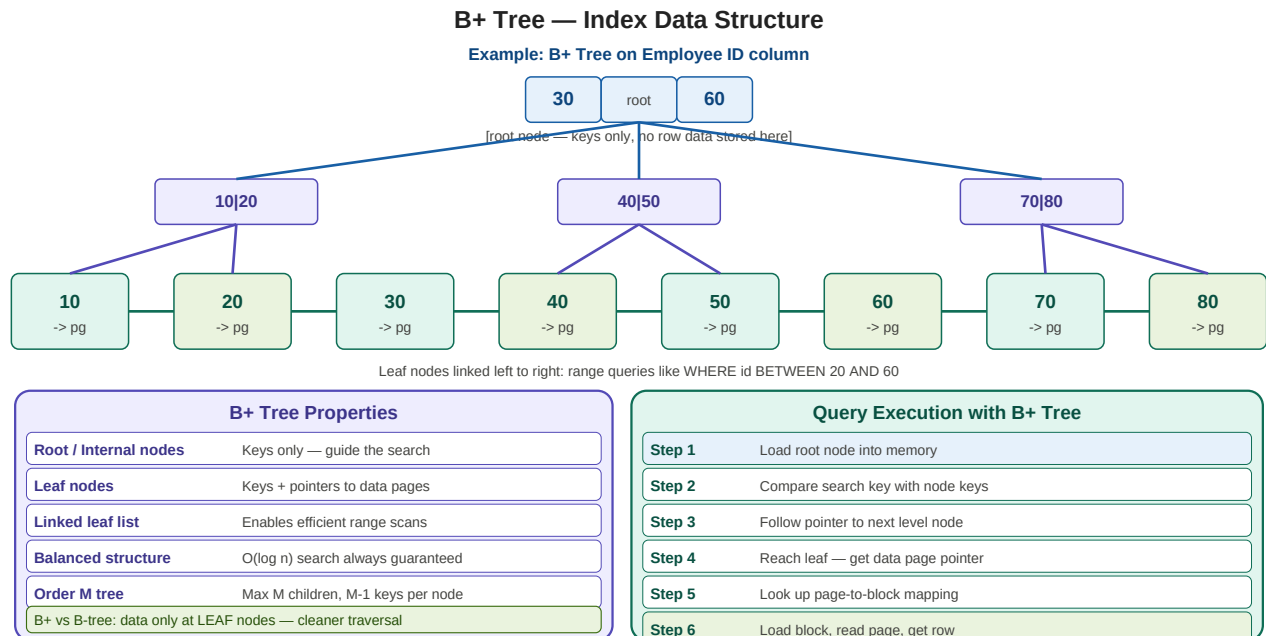
The DBMS maintains a page directory — a mapping of every data page to its physical data block on disk.

DBMS controls pages. The OS and storage system control blocks. DBMS only knows the page-to-block mapping.

Without an index, a query must load ALL 80 pages and scan every row — $O(n)$. With a B+ tree index: $O(\log n)$.

4 & 5. Why Indexing? — B+ Tree Structure

Without an index, every SELECT does a full table scan — reads every page, checks every row. For 1 million rows across 8000 pages this is catastrophically slow. **Indexing** creates a separate sorted B+ tree structure on a column so lookups go from $O(n)$ to $O(\log n)$.



B+ tree: root/internal nodes hold keys only · leaf nodes hold keys + data page pointers · leaves linked for range scans

B-tree vs B+ tree: In a B-tree, data is stored at ALL nodes. In a B+ tree, data is stored **ONLY** at leaf nodes. This makes B+ trees better for disk-based indexes — internal nodes are smaller, more fit in memory, fewer I/Os.

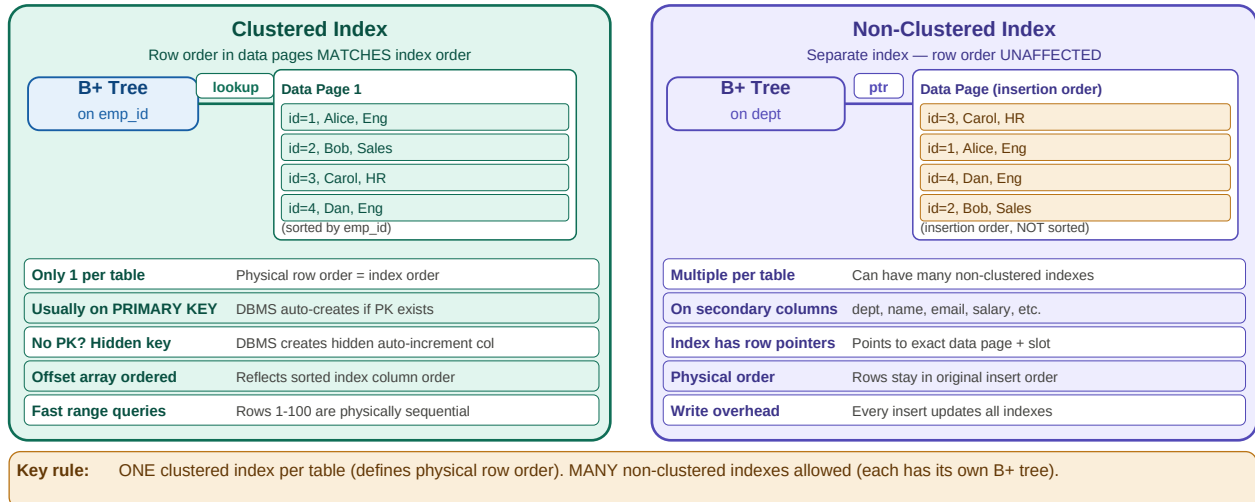
Linked leaf nodes: leaf nodes are connected in a sorted linked list. This makes range queries like WHERE id BETWEEN 100 AND 200 extremely fast — traverse to first leaf, then follow links.

Order M B+ tree: each node can hold M-1 keys and M child pointers. A typical page-sized node (8KB) can hold hundreds of keys — tree height stays very small (3-4 levels) even for billions of rows.

6 & 7. Clustered vs Non-Clustered Index

There are two fundamental types of index. The **Clustered Index** determines the physical order of rows in the data pages — it IS the data. The **Non-Clustered Index** is a separate structure that points to where rows are stored without changing their physical order.

Clustered vs Non-Clustered Index



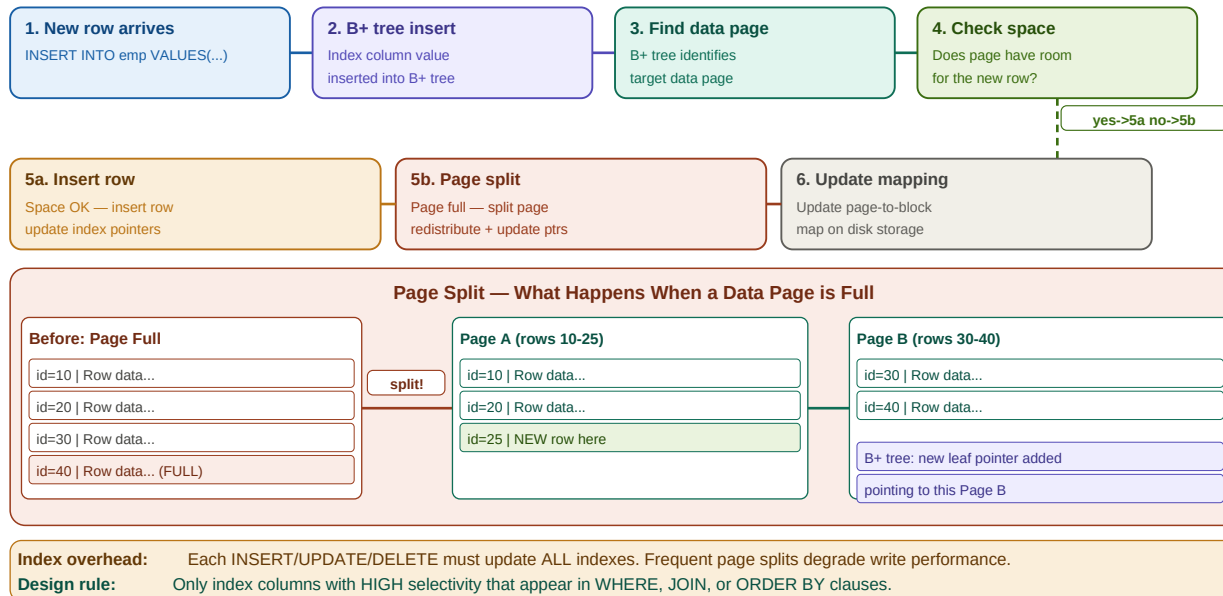
Clustered: rows physically sorted by index column · Non-clustered: separate B+ tree with row pointers · Only 1 clustered allowed

Feature	Clustered Index	Non-Clustered Index
Physical row order	Rows stored in sorted order of index column	Rows in insertion order — unchanged
Count per table	Only ONE allowed	Multiple allowed (but has write cost)
Typical column	Primary key (or hidden auto-increment if none)	Any secondary column: name, dept, email
Data structure	B+ tree IS the table — leaf nodes contain rows	Separate B+ tree with pointers to data pages
Range query speed	Very fast — rows are sequential on disk	Slower — may need many random page reads
Write overhead	Lower — one structure to maintain	Higher — every write updates all NC indexes
Example	SELECT * WHERE id=42 (PK lookup)	SELECT * WHERE dept='Eng' (dept indexed)

8 & 9. Query Execution & Insertion Flow with Page Splits

Query execution with an index involves traversing the B+ tree in memory to find the exact data page, then loading that page from the physical block. **Insertion** involves updating the B+ tree and potentially triggering a page split when a data page is full.

Data Insertion Flow — B+ Tree, Page Split Block Mapping



7-step insertion flow: B+ tree insert -> find page -> check space -> insert or split -> update block mapping

Page Split — What It Means and When It Happens

When a new row must go into a data page that is already full, the DBMS performs a **page split**: the existing page is divided into two pages, rows redistributed, and the B+ tree updated to point to both new pages. This is expensive but necessary.

Stage	What Happens	Performance Impact
Page full detected	New row cannot fit in the target data page	Triggers expensive split operation
Page split	Target page split into two; rows redistributed (roughly half each)	Two I/O writes instead of one
B+ tree update	New leaf node added to index pointing to new page	Tree may need rebalancing (splits can cascade)
Pointer update	All references to the original page updated in index	Adds overhead proportional to index count
Block mapping	New page registered in page-to-block mapping	Small overhead — directory update only

10. NoSQL Indexing — MongoDB, Cassandra, DynamoDB, Redis

RDBMS indexes (B+ tree) work on any column after the fact. In NoSQL, **indexing is inseparable from schema design** — you model your data around your queries because most NoSQL databases only allow efficient access via a limited set of keys.

NoSQL Indexing — MongoDB, Cassandra, DynamoDB, Redis

MongoDB Document DB	Cassandra Wide-Column DB	DynamoDB Managed NoSQL (AWS)	Redis In-Memory Cache/DB
Primary key: <code>_id</code> (B-tree)	Partition key: hashes to node	Partition key: routes to shard	Hash: <code>HSET user:1 name Alice</code>
Single-field: <code>{field: 1}</code>	Clustering key: sort in partition	Sort key: orders in partition	Sorted Set: <code>ZADD scores 100 ali</code>
Compound: <code>{dept:1,salary:1}</code>	PK = partition + clustering	LSI: same partition key	<code>ZRANGEBYSCORE</code> : range on score
Multi-key: indexes arrays	2nd index: low-cardinality only	GSI: alternate partition key	RediSearch: inverted text index
Text index: full-text search	Materialized views: alt access	Sparse: only existing attrs	<code>GEOADD</code> : geo location queries
TTL: auto-expire docs	Design tables for query patterns	GSI enables new access patterns	Bloom filter: membership test
Query example: <code>find({dept:'Eng',salary:{>:80000}})</code> Index used: Compound index on (dept,salary)	Query example: <code>SELECT * WHERE user_id=123 ORDER BY ts</code> Index used: Partition=user_id, cluster=ts	Query example: <code>Query pk=USER#123, sk begins ORDER</code> Index used: GSI on (status, created_at)	Query example: <code>ZRANGEBYSCORE scores 900 +inf</code> Index used: All users with score >= 900

RDBMS: B+ tree on any column. 1 clustered + many non-clustered indexes.

NoSQL: Index = schema design. Model tables around query patterns.

MongoDB: B+tree on any field · Cassandra: partition + clustering key IS the index · DynamoDB: GSI for alternate access · Redis: sorted sets

Database	Index Type	Key Concept	When to Use
MongoDB	B+ tree on any field	Compound indexes follow ESR rule: Equality, Sort, Range	Flexible schema, document access, rich queries
Cassandra	Partition + Clustering key	Partition key = hash for node. Clustering = sort inside partition	Write-heavy, time-series, high throughput
DynamoDB	Partition + Sort key + GSI	GSI gives a completely new partition key for alternate access	Serverless, AWS-native, pay-per-request
Redis	Sorted Sets, RediSearch	ZADD for leaderboards. RediSearch for full-text inverted index	Caching, leaderboards, sessions, pub/sub
Elasticsearch	Inverted index	Every field tokenized and indexed. Shards distribute data	Full-text search, log analytics, aggregations

MongoDB ESR rule: Compound index fields ordered as Equality first, then Sort, then Range (e.g. `{status:1, created_at:1, amount:1}` for `WHERE status=X ORDER BY created_at WHERE amount>Y`).

Cassandra golden rule: one table per query. Cannot do WHERE on a non-partition key without a secondary index (and secondary indexes are expensive in Cassandra).

DynamoDB GSI: creates a completely new B-tree on a different partition key. Reads from GSI are eventually consistent by default. LSI must be defined at table creation time.

Redis Sorted Sets are $O(\log n)$ insert/update and $O(\log n + m)$ range query — perfect for real-time leaderboards and priority queues.

11. Index Overhead, Design Rules & Bonus Interview Concepts

Index Overhead — The Hidden Cost

Indexes are not free. Every index on a table requires its own B+ tree, its own index pages on disk, and must be updated on every INSERT, UPDATE, and DELETE. A table with 10 indexes means every write operation triggers 10 index updates.

Operation	Without Index	With 1 Index	With 5 Indexes
SELECT (point query)	Full scan — $O(n)$	$O(\log n)$ — fast	Same — $O(\log n)$
SELECT (range query)	Full scan — $O(n)$	$O(\log n)$ + sequential reads	Same
INSERT	Write 1 page	Write page + update 1 B+ tree	Write page + update 5 B+ trees
UPDATE	Write 1 page	Write page + update 1 B+ tree if indexed col changed	Update up to 5 B+ trees
DELETE	Write 1 page (mark deleted)	Update 1 B+ tree	Update 5 B+ trees
Page split	N/A	Splits 1 data page, updates 1 B+ tree	Must update all 5 B+ trees

Design Rules — When to Index

- Index columns that appear in WHERE, JOIN ON, or ORDER BY clauses — these are the hot paths.
- High selectivity columns first: gender (2 values) = poor index. user_id (millions of values) = great index.
- Primary key is always the clustered index — DBMS creates it automatically.
- Composite indexes: (dept, salary) helps WHERE dept=X AND salary>Y, but not WHERE salary>Y alone.
- Avoid indexing columns updated frequently — every update must rebuild the index entry.
- Too many non-clustered indexes on a write-heavy table degrades INSERT/UPDATE/DELETE performance badly.
- Use EXPLAIN / query plan to verify the DB is actually using your index before shipping.

Quick Revision

- > Data page: ~8KB unit managed by DBMS. Contains header (96B) + data records (8060B) + offset array (36B).
- > Offset array: pointers to rows inside the page. Gives logical order without physically moving rows.
- > Data block: physical unit managed by OS/disk. DBMS maps data pages to data blocks.
- > Without index: full scan $O(n)$. With B+ tree index: $O(\log n)$ lookup.
- > B+ tree: balanced tree, data only at leaf nodes, leaves linked for range scans.
- > Clustered index: rows physically stored in index order. Only ONE per table. Usually the primary key.
- > No primary key: DBMS creates a hidden auto-increment clustered index column.
- > Non-clustered index: separate B+ tree with pointers to data pages. Multiple allowed.
- > Page split: data page is full -> split into two, redistribute rows, update B+ tree.
- > Index overhead: every INSERT/UPDATE/DELETE must update ALL indexes on that table.
- > High selectivity = good index candidate. Low selectivity (few unique values) = poor candidate.
- > Query execution: load index pages -> traverse B+ tree -> find page pointer -> load block -> read row.

Interview Questions

- > 1. How is table data physically stored in an RDBMS? What is a data page?
- > 2. What are the three sections of a data page and what does each contain?
- > 3. What is the offset array and why is it needed?
- > 4. What is the difference between a data page and a data block?
- > 5. Why is B+ tree preferred over B-tree for database indexing?
- > 6. How do linked leaf nodes in B+ trees help range queries?
- > 7. What is a clustered index? How many can a table have and why?
- > 8. What happens if a table has no primary key?
- > 9. What is a non-clustered index? How is it different from a clustered index?
- > 10. Walk me through a query execution with a non-clustered index step by step.
- > 11. What is a page split? When does it happen and what is the performance impact?
- > 12. A table has 8 indexes and is experiencing slow writes. What is happening and how would you fix it?
- > 13. You need to add an index to improve a slow SELECT. What factors do you consider?
- > 14. What is the difference between RDBMS indexing and NoSQL indexing?
- > 15. How does Cassandra indexing work? Why cant you just WHERE on any column?
- > 16. What is a Global Secondary Index in DynamoDB? When would you use it?
- > 17. You have a MongoDB collection queried by status + date range + amount. Design the index.
- > 18. When would you choose Redis Sorted Sets over a database index for a leaderboard?
- > 19. What is an inverted index? How does Elasticsearch use it?

Key Terms

Data Page	~8KB unit of storage managed by DBMS. Contains header, data records, and offset array.
Offset Array	Array of row pointers inside a data page. Provides logical row ordering.
Data Block	Physical storage unit (4-32KB) managed by OS/disk. DBMS maps pages to blocks.
B+ Tree	Balanced tree index structure. Data only at leaf nodes. Leaves linked for range scans.
Clustered Index	Index where row physical order matches index order. Only 1 per table.
Non-Clustered Index	Separate B+ tree pointing to data pages. Multiple allowed per table.
Page Split	Splitting a full data page into two when a new row cannot fit.
Selectivity	Ratio of unique values in a column. High selectivity = good index candidate.
Full Table Scan	Reading all pages to find matching rows. $O(n)$. Avoided with indexes.
Index Page	Data page storing B+ tree nodes rather than table rows.
WAL	Write-Ahead Log — used for crash recovery during index and data updates.
Composite Index	Index on multiple columns. Column order matters for query efficiency.